🏰 RS004 Week 15 — English Worksheet

Topic: Multi-Level Structure · **Course:** Platformer Game · **Time:** about 45 minutes

This week your game holds **many levels**. Every level is a dictionary of platforms, coins, enemies, and a finish. A `load_level()` function sets up the current one, and reaching the finish loads the **next** — until the last one, which means **victory**. In this worksheet you will read, trace, and explain this code in your own words.

> Keep these words handy: **levels list**, `load_level()`, **next level**, **victory**, **game_state**.

1 · Predict 🔮

Read each snippet. Write what you think it does.

```
LEVELS = [
    { "platforms": [...], "coins": [...], "enemies": [...], "finish": (...) },
    { "platforms": [...], "coins": [...], "enemies": [...], "finish": (...) },
]
```

How many levels are in this list? How would you read the first level's coins?

```
def load_level(idx):
    data = LEVELS[idx]
    platforms = [pygame.Rect(*p) for p in data["platforms"]]
    ...
    return platforms, coins, enemies, finish
```

What does `load_level(1)` set up? Why return four things at once?

```
if player.colliderect(finish):
    current_level += 1
    if current_level < len(LEVELS):
        platforms, coins, enemies, finish = load_level(current_level)
    else:
        game_state = "victory"
```

The player clears the last level. Which branch runs — load or victory?

2 · Trace the Level Flow 🔢

There are 3 levels. The player keeps reaching the finish. Fill in `current_level` and what happens.

```
start:          current_level = 0
clear level:    current_level = ___   →  loads level ___
clear again:    current_level = ___   →  loads level ___
clear again:    current_level = ___   →  game_state = ___
```

3 · Spot the Bug 🐛

Each snippet is broken. Rewrite it so it works, then explain why the original was wrong.

Bug A — Reaching the finish should load the next level. Right now the player just keeps playing the same level forever.

```
if player.colliderect(finish):
    platforms, coins, enemies, finish = load_level(current_level)
```

Hint: `current_level` never advances. Increase it first.

Write the fixed code:

Why was it wrong? Why does your fix work?

Bug B — After the last level the game should show victory. Right now it crashes with an index error when it tries to load a level that doesn't exist.

```
if player.colliderect(finish):
    current_level += 1
    platforms, coins, enemies, finish = load_level(current_level)
```

Hint: check whether `current_level` is still inside the list before loading.

Write the fixed code:

Why was it wrong? Why does your fix work?

Bug C — `load_level` should turn the data into real game objects. Right now the enemies don't move because they were never made into `Enemy` objects.

```
def load_level(idx):
    data = LEVELS[idx]
    platforms = [pygame.Rect(*p) for p in data["platforms"]]
    coins = [pygame.Rect(c[0], c[1], 20, 20) for c in data["coins"]]
    enemies = data["enemies"]
    finish = pygame.Rect(*data["finish"])
    return platforms, coins, enemies, finish
```

Hint: the enemy tuples need to become `Enemy(...)` objects, like the platforms became `Rect`s.

Write the fixed code:

Why was it wrong? Why does your fix work?

4 · Explain the Code 📖

This is a working multi-level loader. Read it carefully, then answer the questions.

```
LEVELS = [
    { "platforms": [...], "coins": [...], "enemies": [...], "finish": (...) },
    { "platforms": [...], "coins": [...], "enemies": [...], "finish": (...) },
    { "platforms": [...], "coins": [...], "enemies": [...], "finish": (...) },
]

def load_level(idx):
    data = LEVELS[idx]
    platforms = [pygame.Rect(*p) for p in data["platforms"]]
    coins = [pygame.Rect(c[0], c[1], 20, 20) for c in data["coins"]]
    enemies = [Enemy(*e) for e in data["enemies"]]
    finish = pygame.Rect(*data["finish"])
    return platforms, coins, enemies, finish

current_level = 0
platforms, coins, enemies, finish = load_level(current_level)

# inside the game loop:
if player.colliderect(finish):
    current_level += 1
    if current_level < len(LEVELS):
        platforms, coins, enemies, finish = load_level(current_level)
    else:
        game_state = "victory"
```

What does `LEVELS` hold, and what does each item in it describe?

Inside `load_level`, why is each enemy wrapped in `Enemy(*e)` instead of left as a tuple?

What does the line `current_level += 1` do, and when does it run?

Explain the `if current_level < len(LEVELS)` check. What does each branch do?

When does `game_state` become `"victory"`?

5 · Explain Your Lesson Code 🎥

Explain the multi-level game **you wrote in today's lesson**. Record a short video on your phone (you can show it running) and walk through your own code using these words: **levels list**, **load_level**, **next level**, **victory**, **game_state**.

> 1. Show your `LEVELS` list and point out one level's data.
> 2. Show `load_level` and explain how it turns data into game objects.
> 3. Clear a level and read the "next level or victory" check out loud.
> 4. Reach the victory screen and explain what `game_state` became.

Write what you will say in your video. Plan it here before you record.

Submit ✅

Send this worksheet + a video explaining your lesson code to teacher on KakaoTalk.